Sessions 6, 7 & 8: Detailed Explanation and Examples

---

1. Introduction to Agile Development Model

The **Agile Development Model** is a flexible and iterative approach to software development. Unlike traditional models (e.g., Waterfall), Agile focuses on delivering small, functional increments of a product frequently, allowing teams to adapt to changes and gather feedback continuously.

Key Features:

- **Iterative Development:** Work is divided into small iterations or cycles.
- **Customer Collaboration:** Active involvement of stakeholders to refine requirements.
- **Flexibility:** Priorities can change based on feedback or new requirements.

Example: Developing an e-commerce website in Agile involves releasing functional components like product listings in one sprint, cart features in the next, and payment options later.

---

2. Agile Development Components

1. **Backlog:**
    - A prioritized list of features or tasks (e.g., "Implement user login").
    - Divided into:
        - **Product Backlog:** All tasks for the entire product.
        - **Sprint Backlog:** Tasks for the current sprint.
2. **Sprints:**
    - Time-boxed cycles (e.g., 2 weeks) where teams focus on completing specific tasks.
3. **Daily Stand-ups:**
    - Short meetings (15 minutes) where team members discuss:
        - What they worked on yesterday.
        - What they plan to do today.
        - Any roadblocks.
4. **Retrospective:**
    - A meeting at the end of a sprint to reflect on what went well and what needs improvement.

---

3. Benefits of Agile

1. **Faster Delivery:**
    - Regular releases ensure quicker deployment of features.
2. **Flexibility:**
    - Adjust to new requirements or feedback without derailing the project.
3. **Improved Collaboration:**

    - Close communication among team members and stakeholders.
4. **Higher Quality:**
    - Continuous testing and feedback reduce defects.

Example: A startup using Agile delivers a minimum viable product (MVP) in 3 months, allowing early user feedback and refinement.

---

4. Introduction to Different Tools Used for Agile Web Development

Agile web development relies on tools to manage tasks, track progress, and facilitate collaboration. Popular tools include:

1. **Jira:** Task and sprint management.
2. **Trello:** Visual task boards.
3. **Asana:** Task assignment and progress tracking.
4. **Slack:** Team communication.
5. **GitHub:** Version control and code collaboration.

Example: Using Jira, a team tracks sprint tasks, assigns responsibilities, and monitors progress in real time.

---

5. Scrum and Extreme Programming (XP)

5.1 Scrum

Scrum is a widely used Agile framework that emphasizes iterative development. Key roles:

1. **Product Owner:** Manages the product backlog and sets priorities.
2. **Scrum Master:** Facilitates the team and ensures adherence to Scrum practices.
3. **Development Team:** Implements tasks during sprints.

Example Workflow:

1. Product Owner creates a prioritized backlog.
2. Team selects tasks for the sprint during the planning meeting.
3. Daily stand-ups monitor progress.
4. Sprint ends with a demo and retrospective.

5.2 Extreme Programming (XP)

XP focuses on improving software quality and responsiveness to changing requirements through practices like:

1. **Pair Programming:** Two developers write code together.
2. **Test-Driven Development (TDD):** Writing tests before the actual code.
3. **Frequent Releases:** Delivering small increments regularly.

Example: A team using XP writes tests for a login feature (e.g., "Check valid and invalid passwords") before coding the functionality.

---

6. Introduction to Atlassian Jira

Jira is a powerful tool for managing Agile projects, offering features like task tracking, sprint planning, and progress monitoring.

6.1 Add Project

To create a project in Jira:

1. Log in to Jira.
2. Click on **Create Project**.
3. Choose a project template (e.g., Scrum, Kanban).
4. Name the project (e.g., "E-commerce Development").
5. Set permissions for the team.

Example: A project named "Mobile App Development" is created using the Scrum template, organizing work into sprints.

6.2 Add Tasks and Sub-Tasks

Tasks: Represent high-level activities (e.g., "Develop login feature"). **Sub-Tasks:** Break a task into smaller steps (e.g., "Create login UI", "Implement authentication logic").

Steps to Add Tasks:

1. Open the project.
2. Click **Create** to add a task.
3. Enter details like the title, description, and assignee.

Example: Task: "Build Payment Gateway" Sub-Tasks:

1. Integrate payment API.
2. Test payment transactions.

6.3 Create Sprints with Tasks

A **sprint** is a time-boxed cycle to complete a set of tasks.

Steps:

1. Go to the project backlog.
2. Drag and drop tasks into the sprint.
3. Start the sprint by clicking **Start Sprint**.
4. Track progress through the sprint board.

Example: Sprint 1 focuses on setting up the application framework and user registration. Tasks include:

- Set up backend.
- Design user registration page.
- Write API for user authentication.

7. Case Study: Developing a Web Application Using Agile Methodology

Scenario:

A team is developing a **Food Delivery Web Application** using Agile.

Step 1: Create Backlog

The Product Owner gathers requirements and creates a backlog:

- User login and registration.
- Restaurant listing.
- Cart and checkout.

Step 2: Plan Sprints

- **Sprint 1:** Build core functionality (login, registration).
- **Sprint 2:** Develop restaurant listing and search.
- **Sprint 3:** Implement cart and payment gateway.

Step 3: Execute Sprints

- Daily stand-ups track progress.
- Tasks are updated in Jira (e.g., "Cart feature: 60% complete").
- At the end of each sprint, a demo is presented to stakeholders.

Step 4: Reflect and Improve

After Sprint 1, the retrospective identifies areas for improvement:

- Allocate more time for code reviews.
- Improve communication between backend and frontend teams.

By following Agile methodology, the team delivers a fully functional food delivery app incrementally, ensuring continuous improvement and satisfaction for stakeholders.